Software engineering

KHOA CÔNG NGHỆ THÔNG TIN
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN

Content

- The Software Engineering Discipline
- The Software Life Cycle
- Software Engineering Methodologies
- Modularity
- Tools of the Trade
- Testing
- Documentation
- Software Ownership and Liability

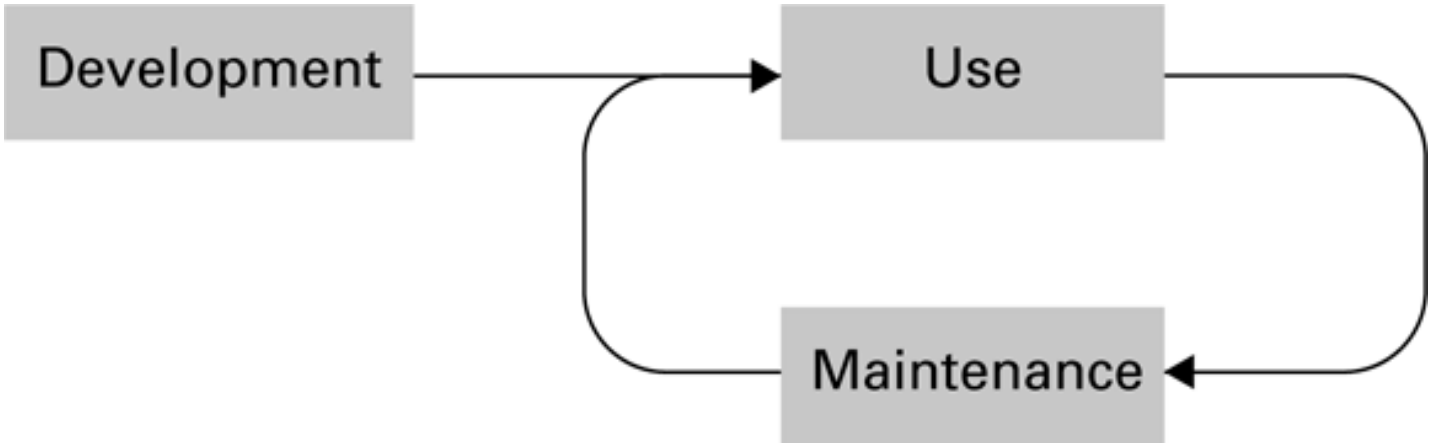
The Software Engineering Discipline

- Distinct from other engineering fields
  - Prefabricated components
  - Metrics
- Practitioners versus Theoreticians
- Professional Organizations: ACM, IEEE, etc.
  - Codes of professional ethics
  - Standards

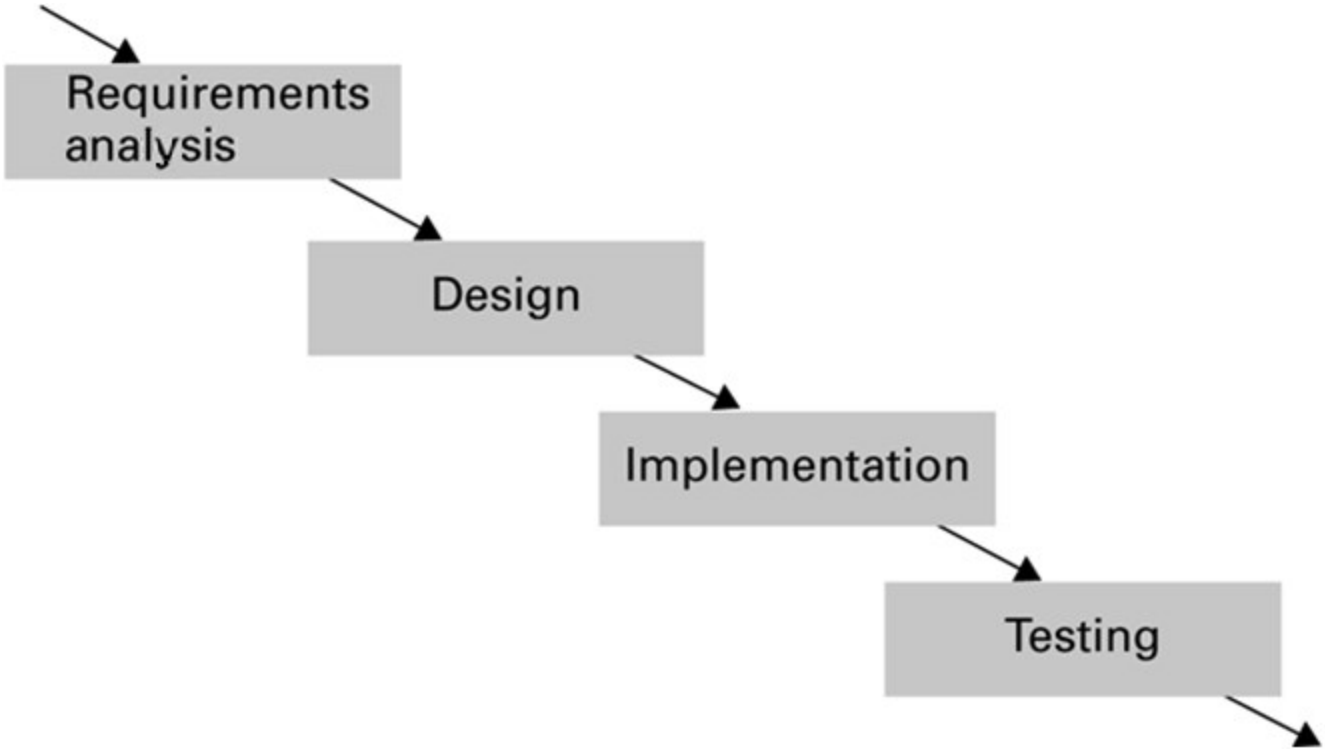
Computer Aided Software Engineering (CASE) tools

- Project planning
- Project management
- Documentation
- Prototyping and simulation
- Interface design
- Programming

The software life cycle



The development phase of the software life cycle



Analysis Stage

- Requirements
  - Application oriented
- Specifications
  - Technically oriented
- Software requirements document

Design Stage

- Methodologies and tools
- Human interface (psychology and ergonomics)

Implementation Stage

- Create system from design
  - Write programs
  - Create data files
  - Develop databases
- Role of “software analyst” versus “programmer”

Testing Stage

- Validation testing
  - Confirm that system meets specifications
- Defect testing
  - Find bugs

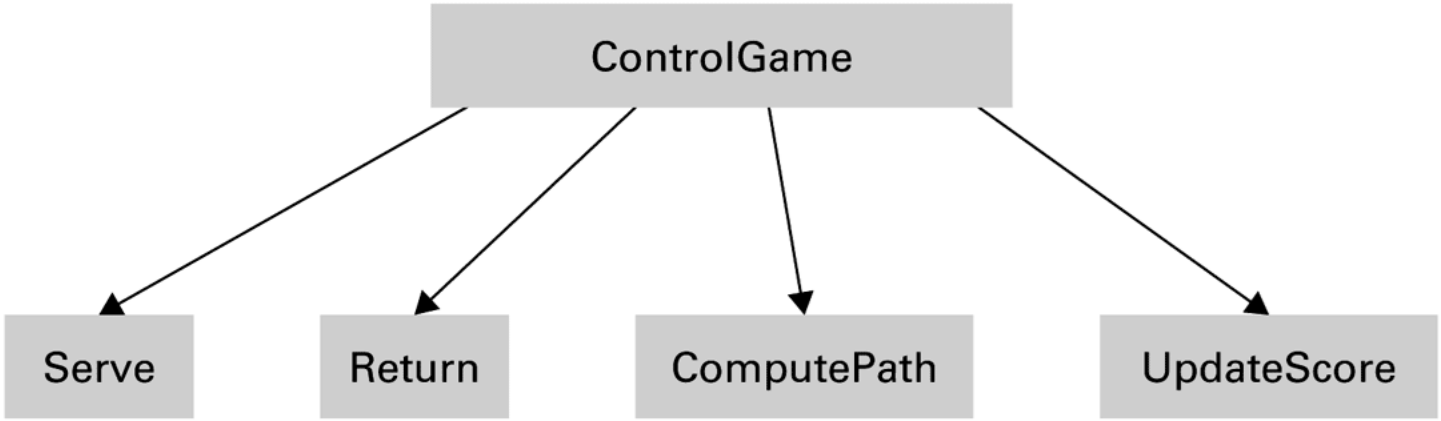
Software Engineering Methodologies

- Waterfall Model
- Incremental Model
  - Prototyping (Evolutionary vs. Throwaway)
- Open-source Development
- Extreme Programming

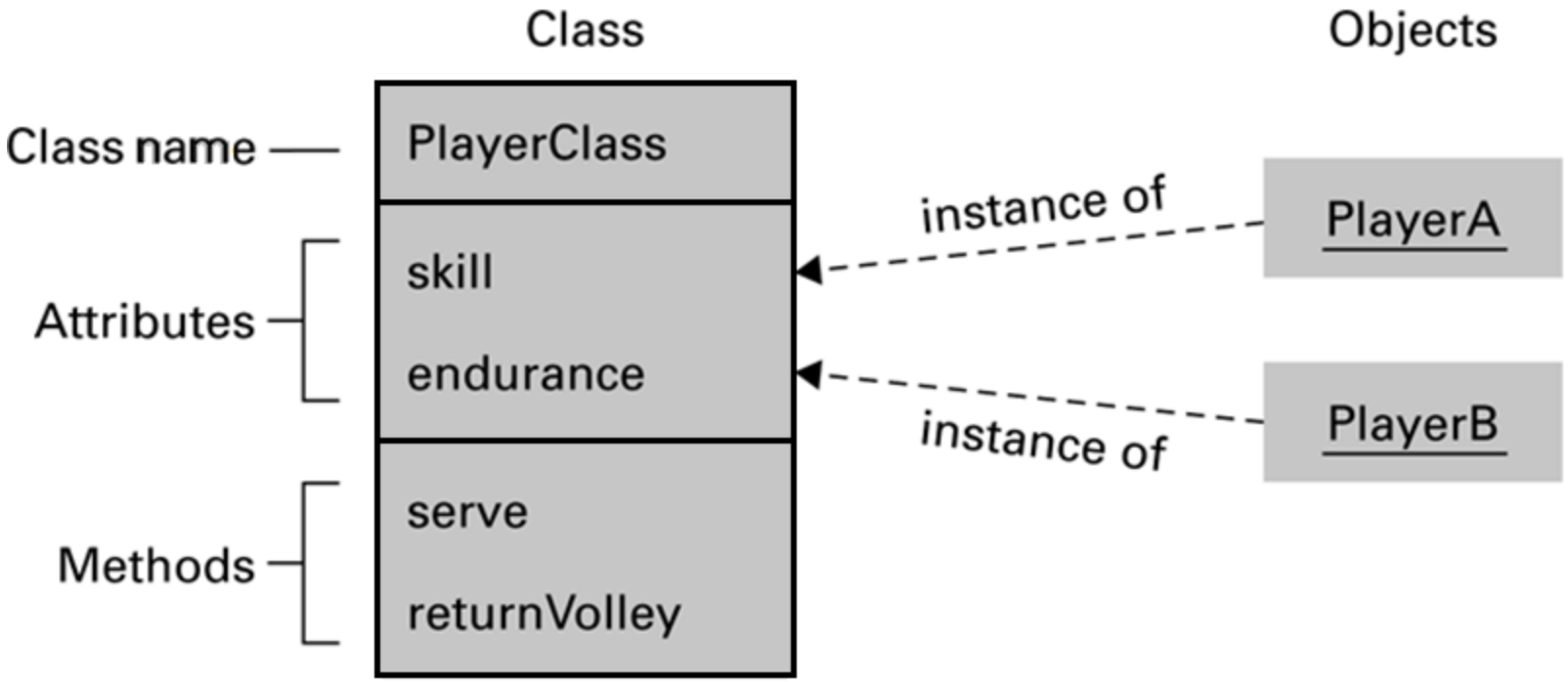
Modularity

- Functions – Imperative paradigm
  - Structure charts
- Objects – Object-oriented paradigm
  - Collaboration diagrams
- Components – Component architecture

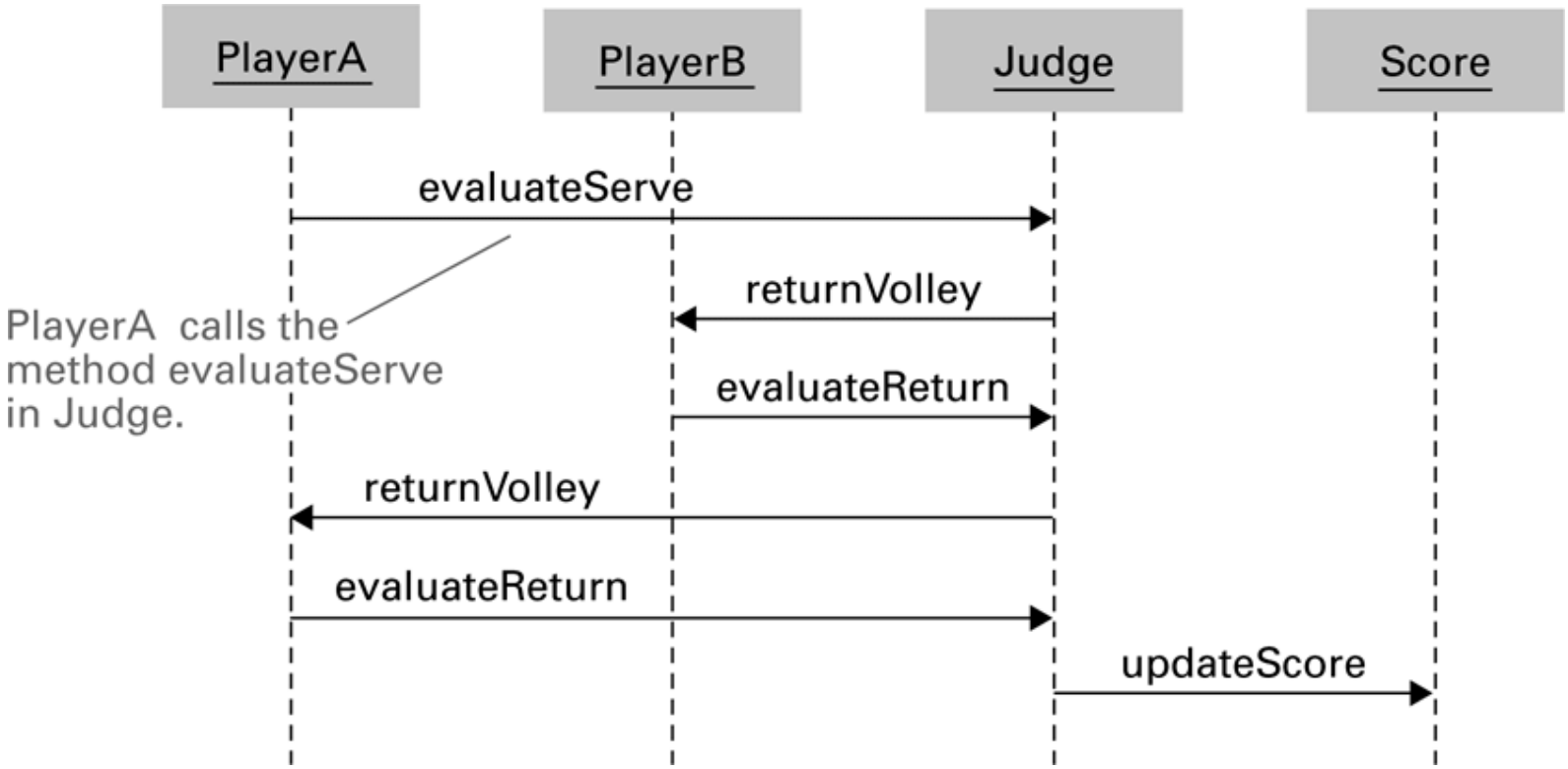
A simple structure chart



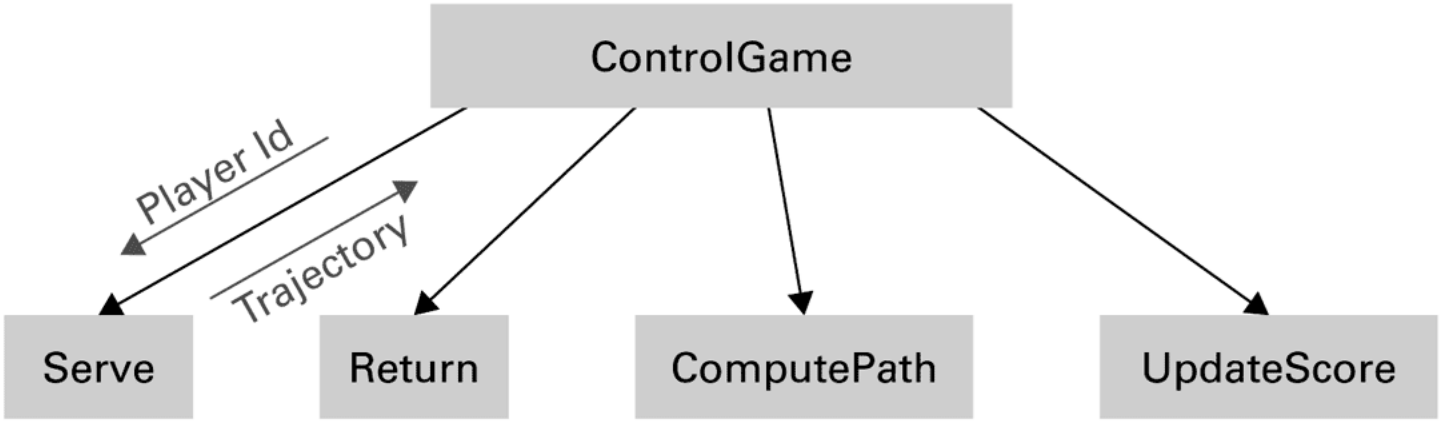
The structure of PlayerClass and its instances



The interaction between objects resulting from PlayerA's serve



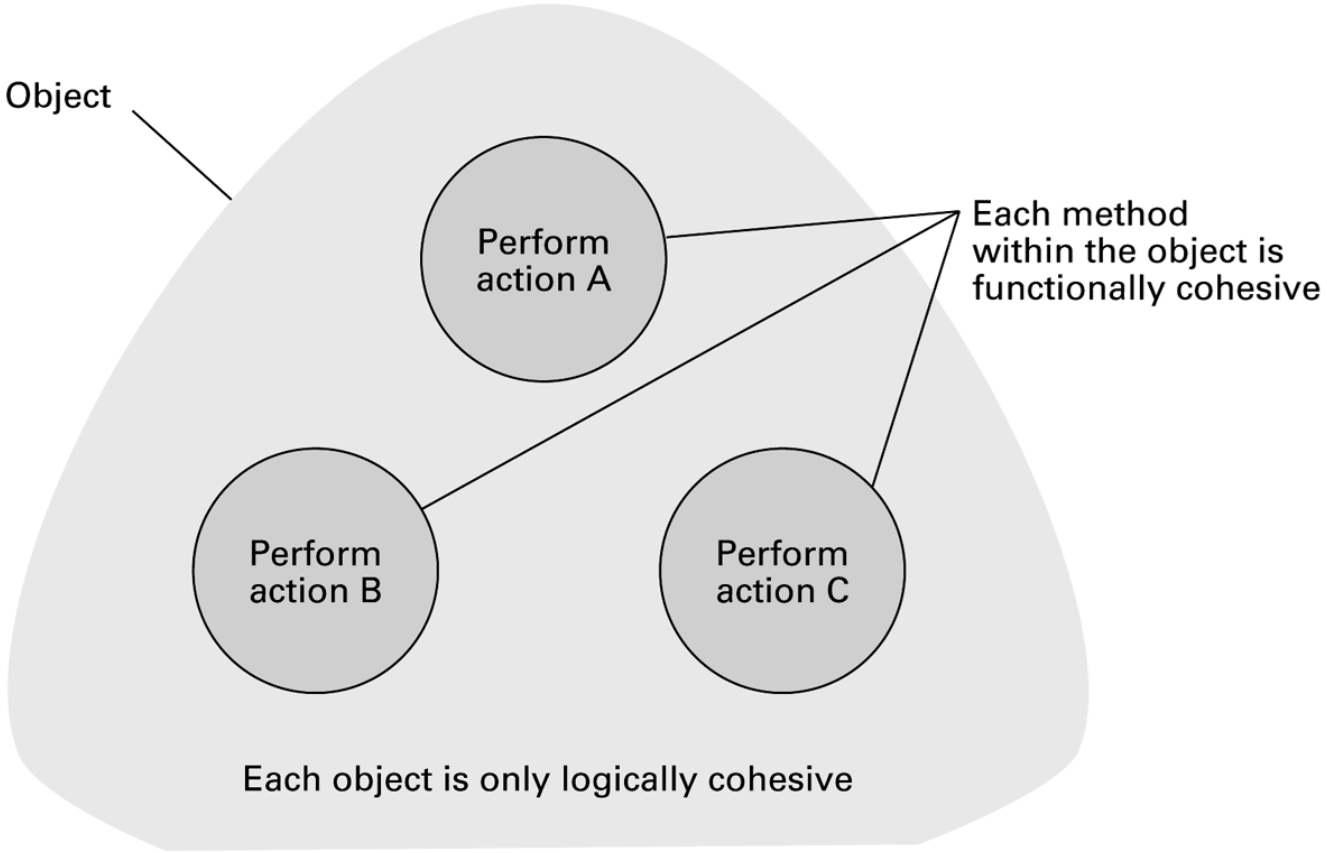
A structure chart including data coupling



Coupling versus Cohesion

- Coupling
  - Control coupling
  - Data coupling
- Cohesion
  - Logical cohesion
  - Functional cohesion

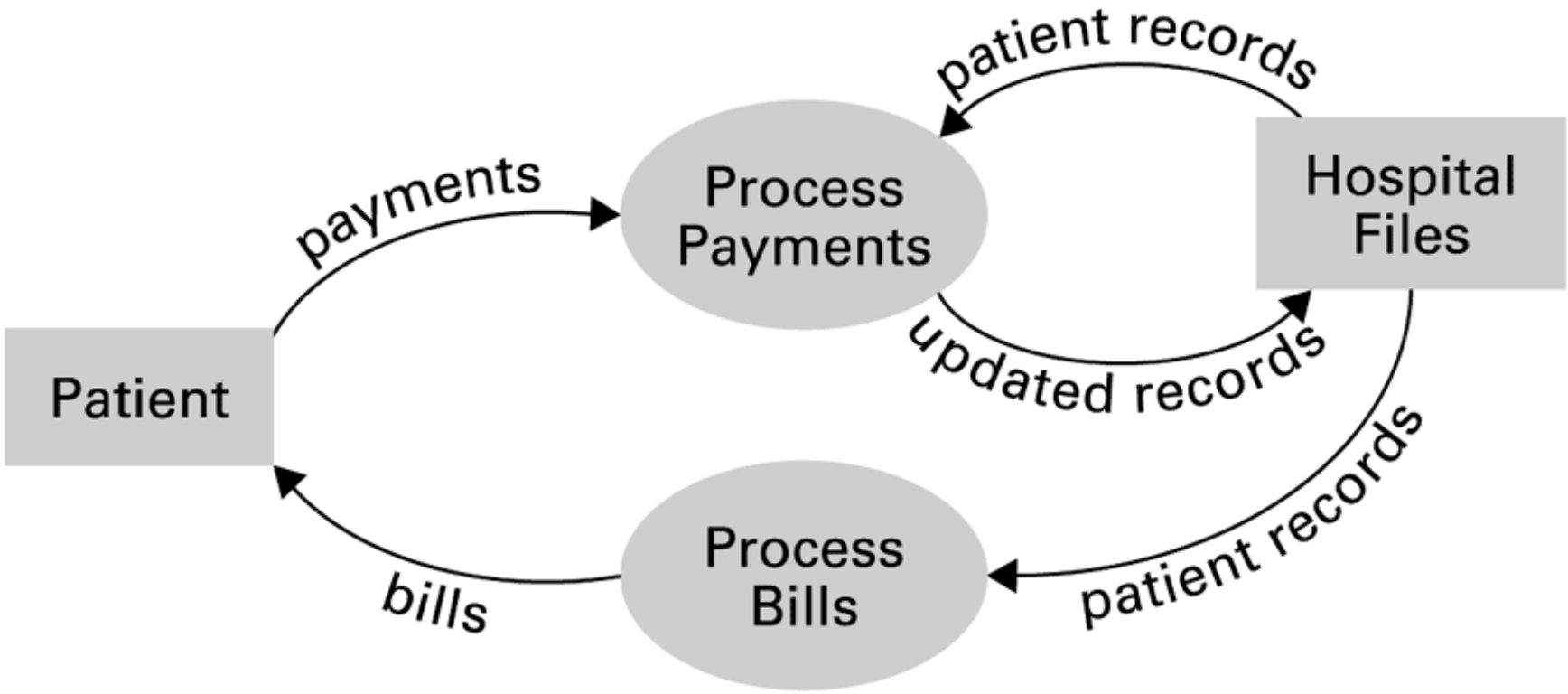
Logical and functional cohesion within an object



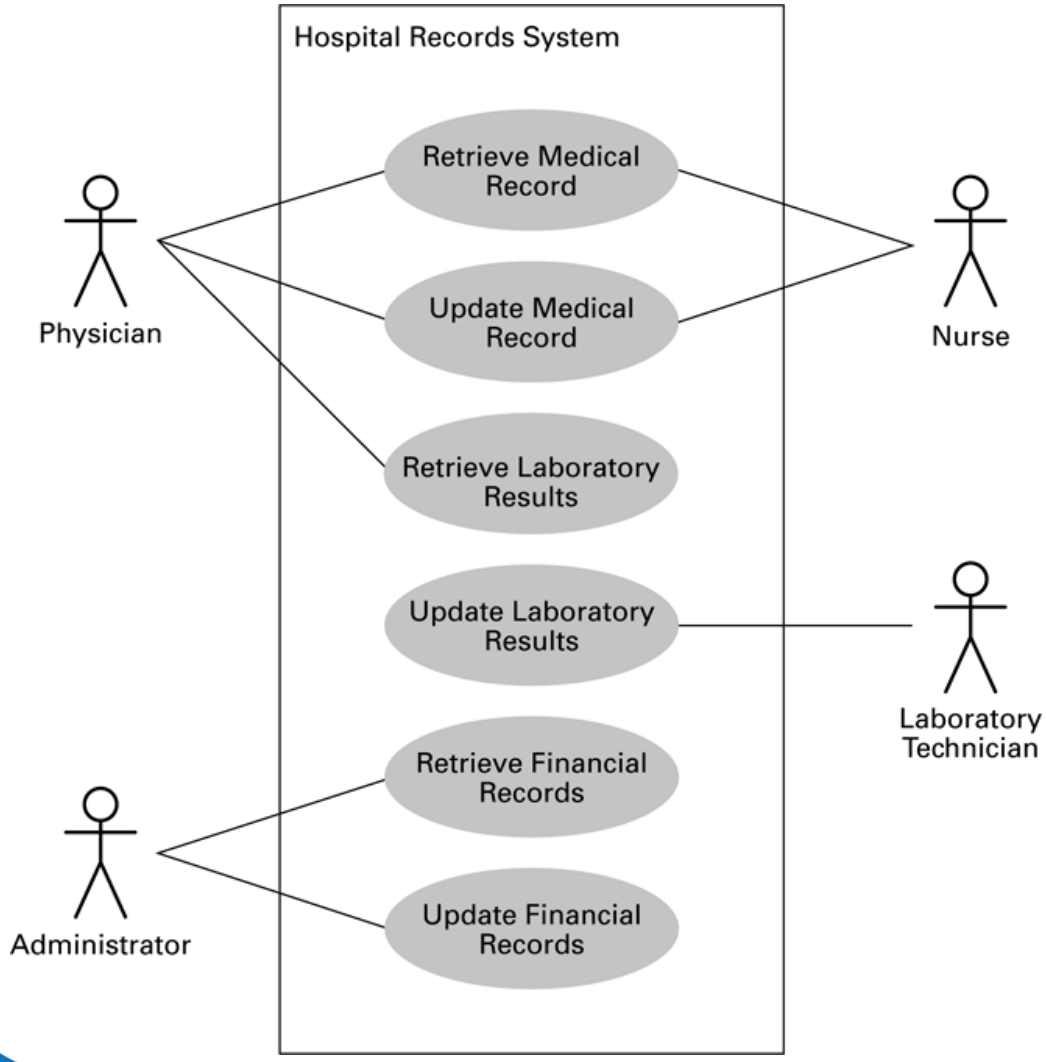
Tools of the Trade

- Data Flow Diagram
- Entity-Relationship Diagram
  - One-to-one relation
  - One-to-many relation
  - Many-to-many relation
- Data Dictionary

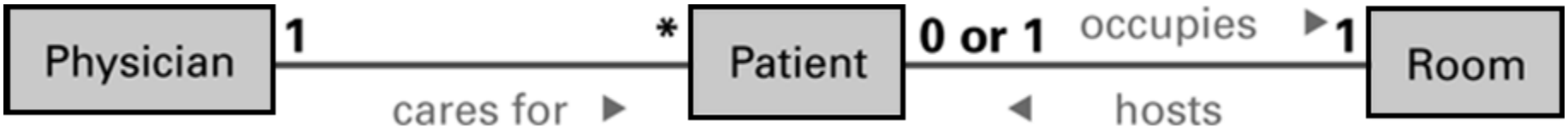
A simple dataflow diagram



A simple use case diagram



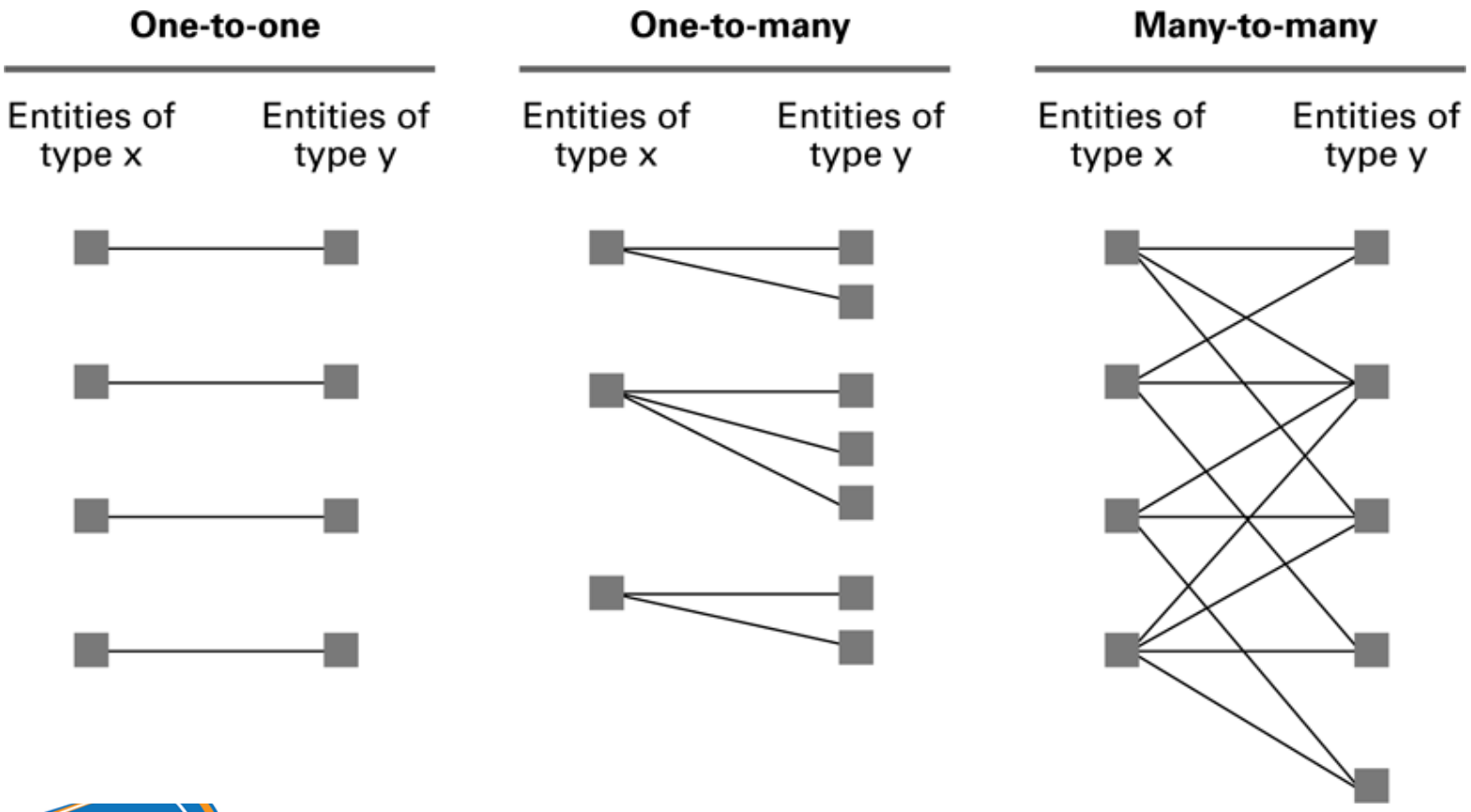
A simple class diagram



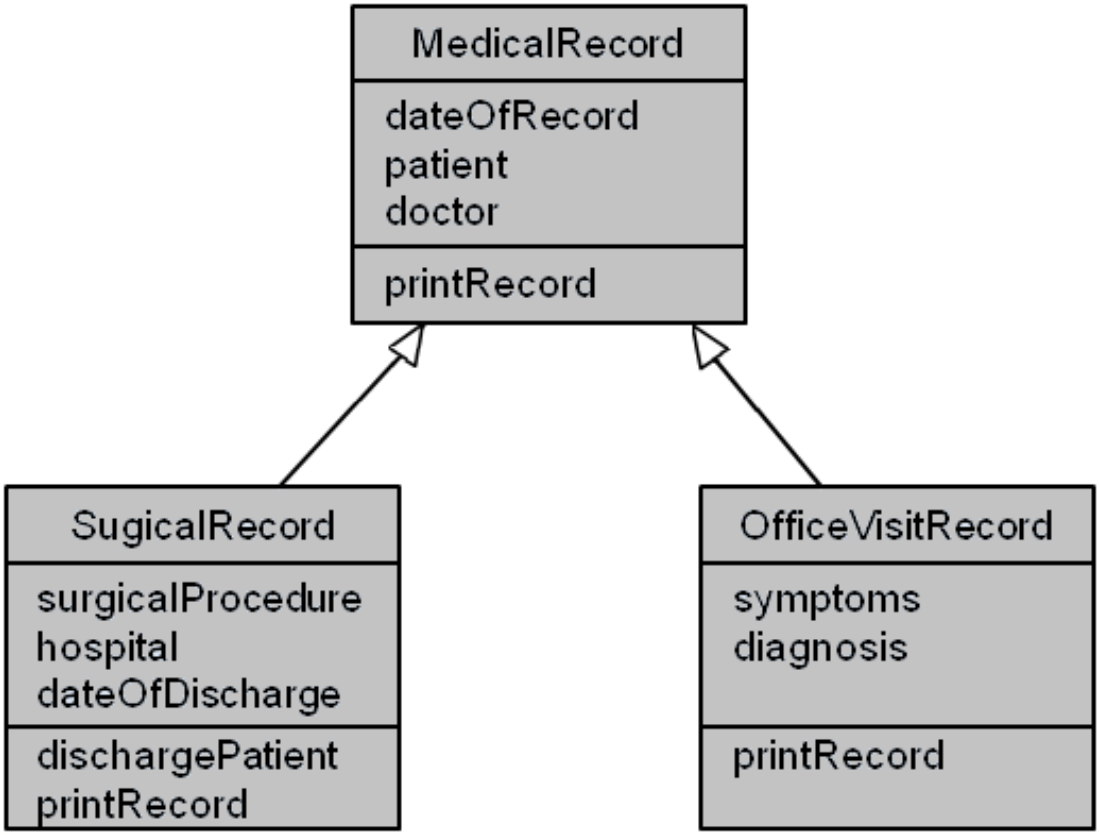
Unified Modeling Language

- Use Case Diagram
  - Use cases
  - Actors
- Class Diagram

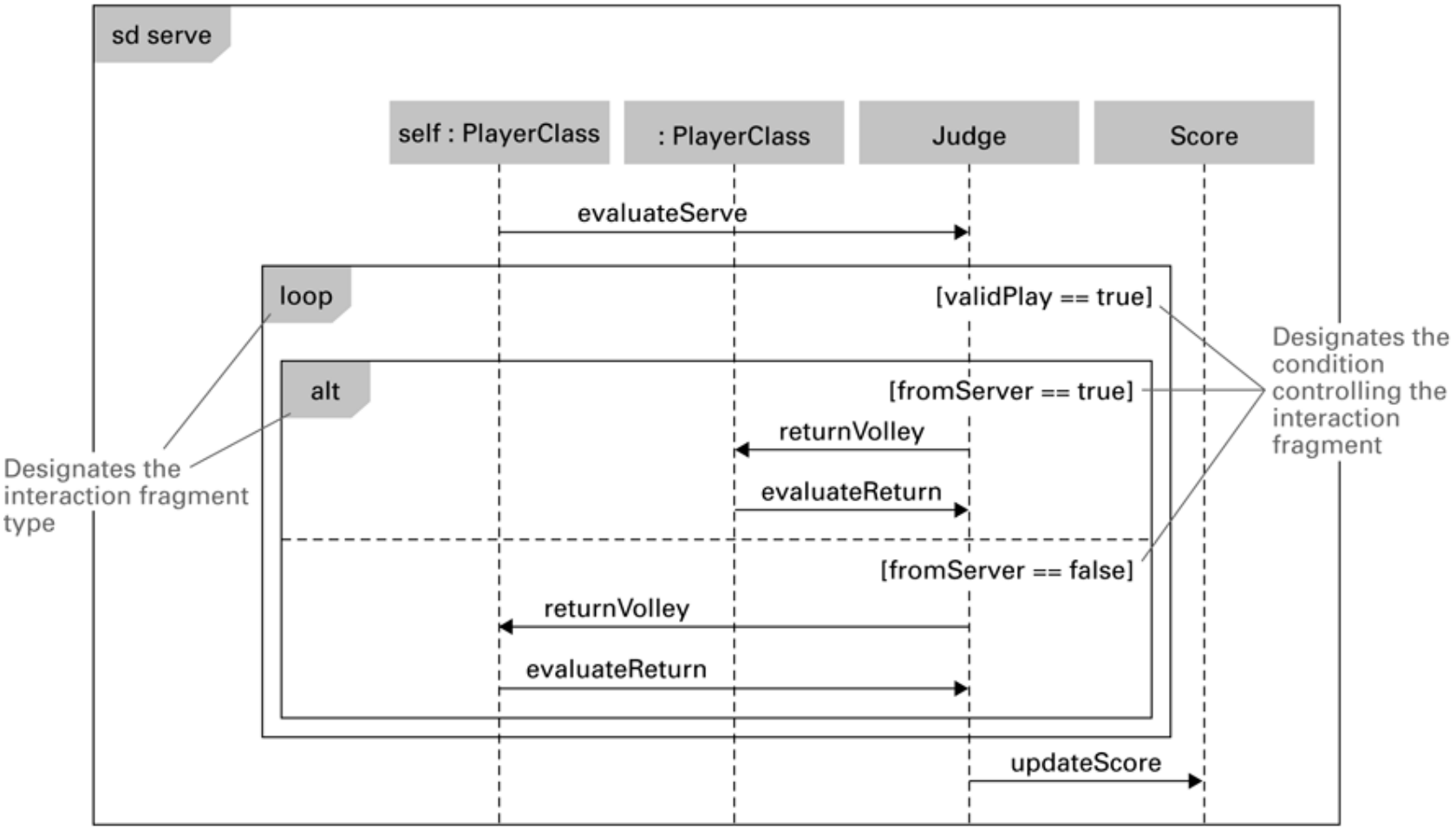
Relationships between entities of types X and Y



A class diagram depicting generalizations



A sequence diagram depicting a generic volley



Structured Walkthoughs

- “Theatrical” experiment
- Class-responsibility-collaboration cards

Design Patterns

- Well designed “templates” for solving recurring problems
- Examples:
  - Adapter pattern: Used to adapter a module’s interface to current needs
  - Decorator pattern: Used to control the complexity involved when many different combinations of the same activities are required
- Inspired by the work of Christopher Alexander in architecture

Software Testing Strategies

- Glass-box testing
  - Pareto principle
  - Basis path testing
- Black-box testing
  - Boundary value analysis
  - Redundancy testing
  - Beta testing

Documentation

- User Documentation
  - Printed book for all customers
  - On-line help modules
- System Documentation
  - Source code
  - Design documents
- Technical Documentation
  - For installing, customizing, updating, etc.

Software Ownership

- Copyright
  - Allow a product to be released while retaining ownership of intellectual property
  - Asserted in all works:
    - Specifications
    - Source code
    - Final product

Software Ownership (continued)

- Software License
  - A legal agreement that grants the user certain permissions without transferring ownership
- Patents
  - Must demonstrate that it is new, usable, and not obvious to others with similar backgrounds
  - Process is expensive and time-consuming

Department of Software Engineering

- Room: I82
- Head of department
  - Dr. Nguyen Van Vu
- Vice head of department
  - Dr. Nguyen Thi Minh Tuyen

Educational objectives

- Ability to analyze requirements, design, test and deploy software systems
- Ability to learn to use software development tools
- Ability to self-study and research new technologies, methods and processes in software industry

Required courses of SE major

- Students accumulate at least 5 courses

Nhập môn công nghệ phần mềm

Quản lý dự án phần mềm

Lập trình Windows

Phát triển ứng dụng Web

Phát triển game

Phân tích và quản lý yêu cầu phần mềm

Phân tích và thiết kế phần mềm

Kiểm chứng phần mềm

Phát triển phần mềm cho thiết bị di động

Optional courses

- Students accumulate at least 5 courses, which contain at least 2 courses (8 credits) of SE department

Các chủ đề nâng cao trong CNPM

Mẫu thiết kế HĐT và ứng dụng

Kiến tập nghề nghiệp (3 tín chỉ)

Khởi nghiệp (2 tín chỉ)

Thiết kế giao diện

Mô hình hóa phần mềm

Kiến trúc phần mềm

Thanh tra mã nguồn

Optional courses (cont.)

Các công nghệ mới trong phát triển PM

Đặc tả hình thức

Lập trình hướng đối tượng nâng cao

Công nghệ **XML** và ứng dụng

Lập trình ứng dụng **Java**

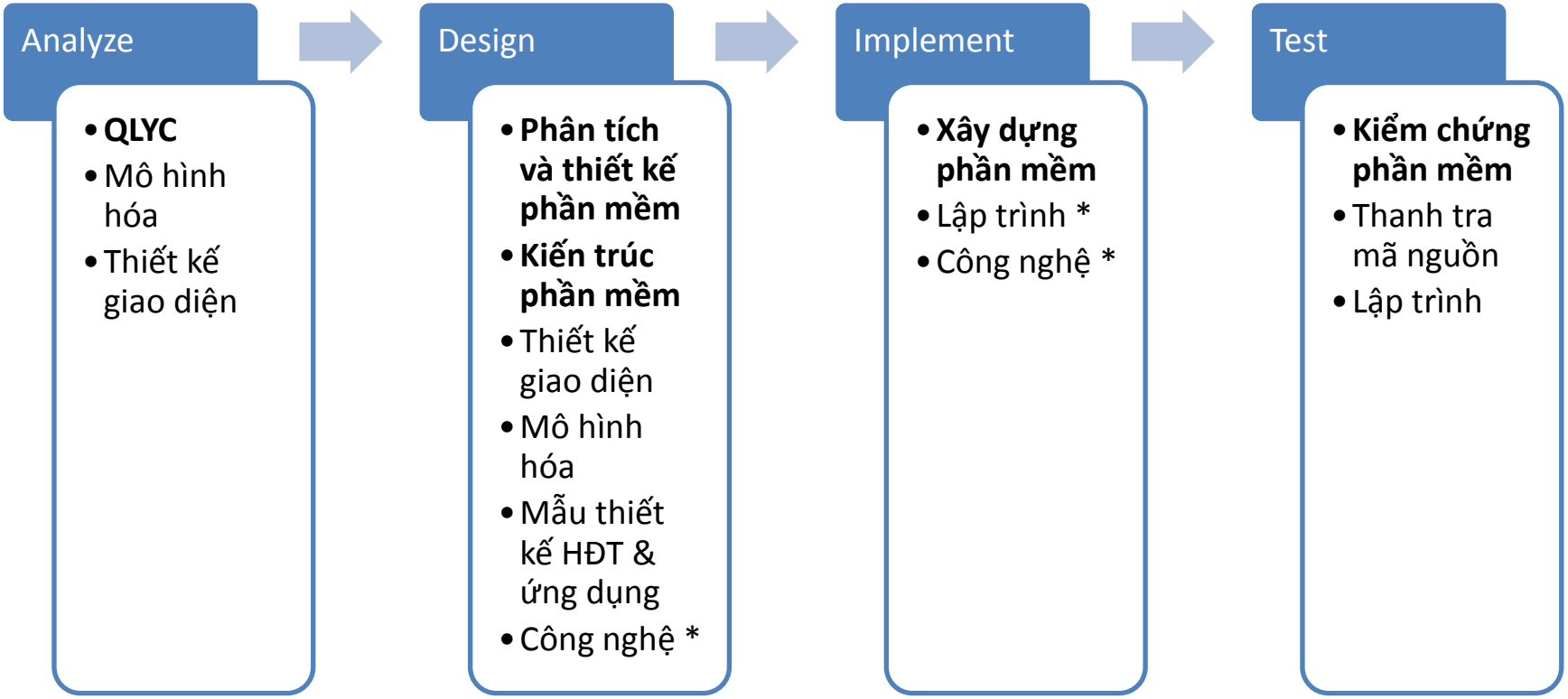
Công nghệ **Java** cho hệ thống **phân tán**

Phát triển phần mềm **nguồn mở**

Phát triển phần mềm cho **hệ thống nhúng**

Đồ án công nghệ PM

Correlation of courses and SE



- **Nhập môn CNPM**
- **Quản lý dự án phần mềm**
- Các chủ đề nâng cao trong CNPM

Career orientation